

Keeping the Table Healthy and Where Hash Tables Run the World

Lecture 18

Data Structures and Algorithms

Where We Left Off

In the last lecture, we covered **Collision Resolution**:

1. **Separate Chaining**: Adding elements to a linked list at the hashed slot.
2. **Open Addressing**: Storing everything directly in the array by probing for empty slots (Linear, Quadratic, Double Hashing).

But a critical question remains:

What happens to performance when the table gets full?

Today we discuss the **Load Factor**, the process of **Rehashing** to keep operations at $O(1)$, and we will explore the immense real-world applications of Hash Tables.

Part 1: Load Factor and Performance

Load Factor Defined

The **Load Factor**, mathematically denoted as α , acts as the "health metric" of our hash table.

$$\alpha = \frac{n}{m} = \frac{\text{Number of elements stored}}{\text{Number of slots in the array}}$$

The Load Factor α directly dictates the performance. It measures how "crowded" our hash table is.

- **Low α** : Super fast operations, but wasted memory. Very few collisions.
- **High α** : Slowly degrading operations, but memory-efficient. Heavy collisions.

Note: For open addressing, α must always be strictly less than 1. For separate chaining, α can theoretically grow as large as you want, though performance will decrease.

Performance vs Load Factor

How exactly does α affect the number of probes (comparisons)?

Load Factor (α)	Chaining (Unsuccessful Search)	Chaining (Successful Search)	Linear Probing (Unsuccessful Search)	Linear Probing (Successful Search)
0.25	0.25	1.125	1.39	1.17
0.50	0.50	1.25	2.50	1.50
0.75	0.75	1.375	8.50	2.50
0.90	0.90	1.45	50.50	5.50

Crucial Observation:

Notice what happens to Linear Probing when α climbs above 0.75. Operations degrade exponentially. Searching an item that doesn't exist at $\alpha = 0.90$ will trigger 50 checks on average!

The Threshold and Rehashing

Because performance falls off a cliff when the table gets crowded, hash tables proactively resize themselves. This is called **Rehashing**.

We define a **maximum load factor threshold**.

- Default in Java's `HashMap` : 0.75
- Default in Python's `dict` : ≈ 0.67

When α exceeds the threshold:

1. We allocate a **new array** that is roughly **double** the size (usually the next prime number).
2. We **re-insert (rehash)** every single element from the old table into the new table using $h(k) = \text{key} \bmod \text{new_m}$.
3. We delete the old table.

The Amortized Cost of Rehashing

Isn't Rehashing $O(n)$ and slow?

Yes. Migrating n elements into a new array takes $O(n)$ time.

However, just like dynamically resizing arrays (e.g., C++ Vectors or Python Lists), this happens very **infrequently**.

The Moving House Analogy:

Imagine your apartment gets too crowded, so you move to an apartment double the size. Moving day is exhausting and expensive! But you only do it once every few years. Most days, you enjoy the spacious rooms.

Because rehashing is rare, the math shows that the **amortized cost** (average cost over many operations) of `insert` still remains strictly **$O(1)$** .

Part 2: Operations Summary

Time Complexity Summary

Operation	Average Case	Worst Case (All Collide)
<code>insert(key, value)</code>	$O(1)$	$O(n)$
<code>search(key)</code>	$O(1)$	$O(n)$
<code>delete(key)</code>	$O(1)$	$O(n)$

The worst case only occurs when **ALL** keys hash to the exact same slot. With a well-designed hash function and rehashing to keep α low, the worst case almost never happens in practice.

Comparison With Other Data Structures

Data Structure	Search	Insert	Delete	Keeps Data Ordered?
Unsorted Array	$O(n)$	$O(1)$	$O(n)$	No
Sorted Array	$O(\log n)$	$O(n)$	$O(n)$	Yes
Balanced BST (e.g. AVL)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	No

The Trade-Off:

Hash tables sacrifice **data ordering** for raw **speed**.

- Need to find minimum, maximum, or query a range of numbers? Use a BST.
- Need to look things up incredibly fast by exact matches? Use a Hash Table.

Part 3: Applications of Hash Tables

Application 1: Dictionaries / Maps

The most fundamental use of hash tables is mapping a key to a value.

- Python's `dict`
- Java's `HashMap`
- C++'s `unordered_map`
- JavaScript's `Map` (and objects!)

Examples:

- *Phone book*: Name (Key) → Phone Number (Value)
- *Student Database*: Roll Number (Key) → Student Record (Value)

Under the hood: Python dicts use a highly optimized open-addressing probing sequence. Java's `HashMap` uses separate chaining, and defensively upgrades chains to Red-Black Trees when lists exceed 8 elements!

Application 2: Symbol Tables in Compilers

Every programming language compiler uses a **Symbol Table** -- a fast hash table mapping identifier names to their attributes.

When you type: `int count = 5;`

The compiler stores: `Name = "count", Type = int, Scope = local, Value = 5`

Why?

During compilation, looking up variables happens thousands of times a second:

- Scanner: Is `if` a keyword or variable? ($O(1)$ hash check)
- Semantic Analysis: Is `count` declared? Are types matched? ($O(1)$ hash check)

Without hash tables, compiling modern software with millions of lines of code would take hours or days due to $O(n)$ slow searches!

Application 3: Caching

Caches remember recent results so we do not have to recompute or re-download them. Because caching is about speed, lookups **must** be $O(1)$.

Where it is used:

- **Web Browsers:** Caches images and webpages. URL (**key**) mapped to Page Content (**value**).
- **DNS Caching:** Translating `www.google.com` (**key**) to an IP address like `142.250.190.4` (**value**).
- **Database Query Caches:** Complex SQL query string (**key**) mapped directly to the queried data (**value**).

Application 4: Counting & Sets

Frequency Analysis

Hash tables are perfect for counting things!

- Finding word frequencies in a book: Word (`key`) mapped to Count (`value`).
- Checking an array for duplicates quickly: Insert elements into a hash table and check if they already exist. $O(n)$ total time!

Sets

A **Set** is just a Hash Table but with no "values" -- only the "keys" matter.

- **Python's** `set` , **Java's** `HashSet`
- Gives you mathematical set operations: Membership testing $O(1)$, Union, Intersection, and Difference.
- Example: *Find common friends between two social media users.* Store User A's friends in a `HashSet`, then check User B's friends for membership in $O(1)$ time.

Final Takeaways & Graph Transition

Hash Tables: The Big Picture

Hash tables are arguably the most practically important data structure in the entire computer science ecosystem. They power your databases, internet routers, browsers, and the very compilers you run your code on.

The Golden Rule: Choose hash tables when you need blindingly fast $O(1)$ lookups, but be prepared to guarantee a good hash function and manage rehashing.

Transition to Graphs

Up until now, the data structures we've covered follow rigid structures:

- **Arrays & Linked Lists:** Linear lines.
- **Trees:** Hierarchical, "parent-child" one-way relationships.

What about the real world?

- A road network. (Multiple paths between two cities).
- The internet. (Webpages linking forming loops and webs).
- Social Networks. (Alice is friends with Bob and Bob with Alice -- no "parent-child").

In the next lecture, we break out of hierarchies completely. We introduce **Graphs**, the ultimate and most generalized data structure for modeling relationships!